

Trabalho Prático 1 (TP1) - Lexer JSON e CSV

Integrantes: João Oliveira Galeazzi, Mateus Simões Neubarth, Augusto Baldino e
Guilhermo Herter

Vídeo: <https://youtu.be/oKRGFrbULa0>

1. Análise e Comparação:

Regras JSON:

STRING (texto delimitador)

Strings são sequências de caracteres delimitadas obrigatoriamente por aspas duplas (“”). Caracteres como aspas duplas e barra invertida são considerados especiais, podendo ser representados dentro das aspas duplas com \, como \", \\.

NUMBER (número)

Números podem conter um sinal negativo, uma parte inteira obrigatória e uma parte decimal como opcional, representado com um “.”.

WHITESPACE (espaços em branco)

Espaços em branco, como espaço e quebra de linha) podem aparecer livremente entre os elementos do arquivo JSON e não afetam a estrutura do objeto.

Regras

CSV:

O analisador léxico do CSV foca na estruturação de dados tabulares. Identificamos três regras fundamentais que regem a estrutura dos dados:

TEXT (texto simples)

definição de um texto é qualquer sequência de caractere com exceção de vírgula, quebra de linha ou aspas duplas (~[,/n/r]).

STRING (Texto delimitado)

Strings são identificadas como qualquer sequência de caracteres entre aspas duplas, com uma regra a mais que não pode conter uma aspa dupla (“) sozinha.

SEPARADOR (delimitador)

Regra responsável por identificar a vírgula (,), que atua como o divisor crítico entre as colunas de dados.

Comparação das regras dos dois lexers

Tanto o CSV quanto o JSON utilizam aspas duplas para delimitar o texto e permitir que o que estiver entre elas será tratado como conteúdo. A Principal diferença é que o JSON é mais rigoroso, em que toda a String deve ser definida entre aspas, já o CSV torna isso opcional.

O CSV possui uma regra mais permissiva, pois TEXT aceita praticamente qualquer caractere, que não seja delimitador. Já o JSON é mais rigoroso, separando explicitamente `STRING` e `NUMBER`.

No JSON, espaços em branco (WHITESPACE) são ignorados pelo analisador léxico e não afetam a estrutura. No CSV, a vírgula (SEPARADOR) é essencial, pois define a divisão entre colunas. Assim, enquanto o whitespace no JSON é apenas estético, o separador no CSV é estrutural e indispensável.

Uso de regras

2. Demonstração de Entradas:

STRING (texto delimitador)

JSON válido 01:

The screenshot shows a web-based JSON parser interface. At the top, the 'Input' field contains the text 'create-table.hql' and a status indicator '(#0 Type STRING Line 1:0)'. Below the input field, a text area displays the JSON string: `1 "texto com aspas"`. Underneath the text area, there are buttons for 'Start rule', 'Run', and 'Show profiler'. At the bottom, a 'Tree Hierarchy' section shows a parse tree with the root node 'json:1' branching into two children: the string literal '"texto com aspas"' and the end-of-file token '<EOF>'. The 'json:1' node is highlighted in blue.

JSON válido 02:

Input `create-table.hql` (#0 Type STRING Line 1:0)

```
1 "valor com \"aspas\" internas"
```

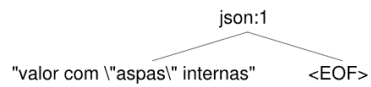
Start rule

json

Run

Show profiler

Tree Hierarchy



JSON inválido 01:

```
1 'aspas simples'
```

Start rule

json

Run

Show profiler

Parser console

```
1:0 token recognition error at: ''
1:1 token recognition error at: 'a'
1:2 token recognition error at: 's'
1:3 token recognition error at: 'p'
1:4 token recognition error at: 'a'
1:5 token recognition error at: 's'
1:7 token recognition error at: 's'
1:8 token recognition error at: 'i'
1:9 token recognition error at: 'm'
1:10 token recognition error at: 'p'
1:11 token recognition error at: 'l'
1:12 token recognition error at: 'e'
1:13 token recognition error at: 's'
1:14 token recognition error at: ''
```

Tree Hierarchy

```
graph TD
    json1["json:1"] --- eof["<EOF>"]
```

JSON inválido 02:

1

"texto sem fechar"

Start rule ?

json

Run

Show profiler

Parser console

1:0 token recognition error at: '"texto sem fechar'

Tree Hierarchy

json:1

<EOF>

NUMBER (número)

JSON válido 01:

Input create-table.hql ? (#0 Type NUMBER Line 1:0)

1

-123.45

Start rule ?

json

Run

Show profiler

Tree Hierarchy

json:1

-123.45 <EOF>

JSON válido 02:

Input `create-table.hql` ? (#0 Type NUMBER Line 1:0)

```
1 10e+5
```

Start rule ?

`json` Run Show profiler

Tree Hierarchy

```

graph TD
    json1["json:1"] --> 10e+5["10e+5"]
    json1 --> EOF1["<EOF>"]
  
```

JSON inválido 01:

Input `create-table.hql` ?

```
1 1.
```

Start rule ?

`json` Run Show profiler

Parser console

```
1:1 token recognition error at: '.'
```

Tree Hierarchy

```

graph TD
    json1["json:1"] --> 1["1"]
    json1 --> EOF2["<EOF>"]
  
```

JSON inválido 02:

1 02

Start rule ?

`json` Run Show profiler

Parser console

```
1:0 token recognition error at: '0'
```

Tree Hierarchy

```

graph TD
    json1["json:1"] --> 2["2"]
    json1 --> EOF3["<EOF>"]
  
```

WHITESPACE (espaços em branco)

JSON válido 01:

Input ?

1

Start rule ?

json

Run

Show profiler

Tree Hierarchy

json:1

<EOF>

JSON válido 02:

Input ?

1 "abc"

2 "def"

Start rule ?

json

Run

Show profiler

Tree Hierarchy

json:1

"abc"

"def"

<EOF>

JSON inválido 01:

1 a

Start rule ?

json

Run

Show profiler

Parser console

1:0 token recognition error at: 'a'

Tree Hierarchy

json:1

<EOF>

JSON inválido 02:

```
input | state <state> | ... | <eof>
1 b
1:0 token recognition error at: 'b'
```

Start rule [?](#)
json Run Show profiler

Parser console
1:0 token recognition error at: 'b'

Tree Hierarchy
json:1
|
<EOF>

TEXT (texto simples)

JSON válido 01:

Input example1.csv

1 0345391802
2
3
4

Start rule csvFile Run Show profiler

Tree Hierarchy
csvFile:1
├── hdr:1
│ └── row:1
│ └── field:1
│ └── 0345391802
├── row:1
│ ├── field:3
│ │ └── \n
│ ├── field:3
│ │ └── \n
│ └── field:3
│ └── \n
└── row:1 <EOF>

JSON válido 02:

Input example1.csv (#2 Type null Line 2:0)

1 18.99

2

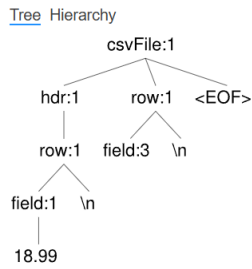
3

Start rule

csvFile

Run

Show profiler



JSON inválido 01:

Input example1.csv

1

2

3

Start rule

csvFile

Run

Show profiler

Parser console

1:0 token recognition error at: ""

1:1 mismatched input "" expecting {',', '\n', '\n'}

Tree Hierarchy

```
graph TD
    csvFile1["csvFile:1"] --> hdr1["hdr:1"]
    csvFile1 --> row1_1["row:1"]
    csvFile1 --> eof1["<EOF>"]
    row1_1 --> row1_2["row:1"]
    row1_1 --> eof2["<EOF>"]
    row1_2 --> field3["field:3"]
    row1_2 --> eof3["<EOF>"]
    field3 --> eof4["<EOF>"]
```

JSON inválido 02:

Input ?

```

1  "
2  abc

```

Start rule ?

Parser console

```

1:0 token recognition error at: '"\r\nabc'
2:3 mismatched input ' ' expecting {' ','\n','\n'}

```

Tree Hierarchy

```

graph TD
    csvFile:1 --> hdr:1
    csvFile:1 --> row:1
    csvFile:1 --> EOF1["<EOF>"]
    row:1 --> row:1_1
    row:1 --> field:3
    row:1 --> EOF2["<EOF>"]
    row:1 --> EOF3["<EOF>"]
    row:1_1 --> field:3_1
    row:1_1 --> EOF4["<EOF>"]
    field:3_1 --> field:3_2
    field:3_1 --> EOF5["<EOF>"]
    field:3_2 --> field:3_3
    field:3_2 --> EOF6["<EOF>"]

```

STRING (Texto delimitado)

JSON válido 01:

Input ?

```

1  "Douglas Adams "
2
3  |

```

Start rule ?

Tree Hierarchy

```

graph TD
    csvFile:1 --> hdr:1
    csvFile:1 --> row:1
    csvFile:1 --> EOF1["<EOF>"]
    row:1 --> row:1_1
    row:1 --> field:3
    row:1 --> LF1["\n"]
    row:1_1 --> field:2
    row:1_1 --> LF2["\n"]
    field:2 --> DouglasAdams["\"Douglas Adams \""]

```

JSON válido 02:

Input ?

```

1  "Timothy ""The Parser"" Campbell"
2
3  |
4

```

Start rule ?

Tree Hierarchy

```

graph TD
    csvFile:1 --> hdr:1
    csvFile:1 --> row:1
    csvFile:1 --> row:2
    csvFile:1 --> EOF1["<EOF>"]
    row:1 --> row:1_1
    row:1 --> field:3
    row:1 --> LF1["\n"]
    row:1_1 --> field:2
    row:1_1 --> LF2["\n"]
    field:2 --> Timothy["\"Timothy \""]
    row:2 --> field:3_2
    row:2 --> LF3["\n"]
    field:3_2 --> TheParser["\"The Parser\""]

```

JSON inválido 01:

Input ?

```

1 "Douglas Adams
2
3

```

Start rule ?

Parser console

```

1:0 token recognition error at: '"Douglas Adams\n\n'
3:0 mismatched input '' expecting {'', '\r', '\n'}

```

Tree Hierarchy

```

graph TD
    csvFile1["csvFile:1"] --> hdr1["hdr:1"]
    csvFile1 --> row1_1["row:1"]
    csvFile1 --> eof1["<EOF>"]
    hdr1 --> row1_2["row:1"]
    row1_1 --> field3["field:3"]
    row1_1 --> eof2["<EOF>"]
    row1_1 --> eof3["<EOF>"]
    row1_2 --> field3_1["field:3"]
    row1_2 --> eof4["<EOF>"]

```

JSON inválido 02:

Input ?

```

1 Timothy "The Parser" Campbell
2
3

```

Start rule ?

Parser console

```

1:8 mismatched input '"The Parser"' expecting {'', '\r', '\n'}
1:20 extraneous input ' Campbell' expecting {'', '\r', '\n'}

```

Tree Hierarchy

```

graph TD
    csvFile1["csvFile:1"] --> hdr1["hdr:1"]
    csvFile1 --> row1_1["row:1"]
    csvFile1 --> row1_2["row:1"]
    csvFile1 --> eof1["<EOF>"]
    hdr1 --> row1_3["row:1"]
    row1_1 --> field2["field:2"]
    row1_1 --> nl1["\n"]
    row1_2 --> field3["field:3"]
    row1_2 --> nl2["\n"]
    row1_3 --> field1["field:1"]
    row1_3 --> tp1["The Parser"]
    field1 --> timothy["Timothy"]
    field2 --> campbell["Campbell"]
    field3 --> tp2["The Parser"]

```

SEPARADOR (delimitador)

JSON válido 01:

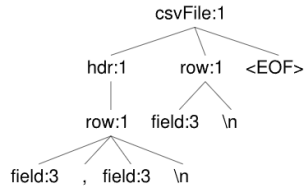
Input ?

```
1 ,
2
3 |
```

Start rule ?

csvFile **Run** Show profiler

[Tree](#) Hierarchy



JSON válido 02:

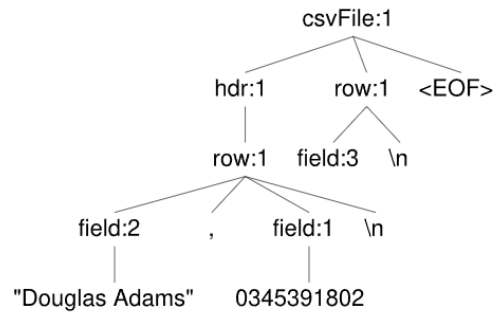
Input ?

```
1 "Douglas Adams",0345391802
2
3 |
```

Start rule ?

csvFile **Run** Show profiler

[Tree](#) Hierarchy



JSON inválido 01:

Input (#1 Type null Line 1:1)

1 ;
2

2:0 mismatched input '<EOF>' expecting {'', '\r', '\n'}

Start rule

Parser console

2:0 mismatched input '' expecting {'', '\r', '\n'}

Tree Hierarchy

csvFile:1

hdr:1

row:1

<EOF>

row:1

field:3

<EOF>

field:1

\n

;

JSON inválido 02:

Input (#3 Type null Line 2:0)

1 "Douglas Adams";0345391802
2
3

1:15 extraneous input ';0345391802' expecting {'', '\r', '\n'}

Start rule

Parser console

1:15 extraneous input ';0345391802' expecting {'', '\r', '\n'}

Tree Hierarchy

csvFile:1

hdr:1

row:1

<EOF>

row:1

field:3

\n

field:2

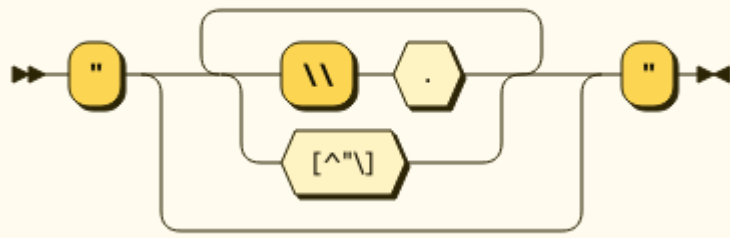
:0345391802

\n

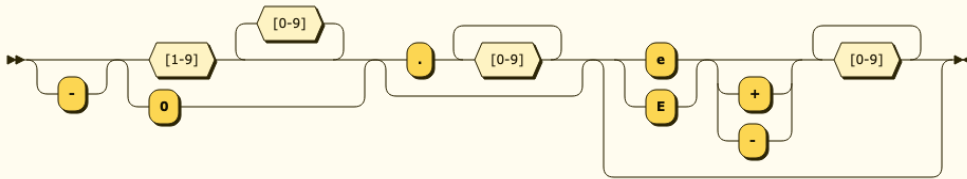
"Douglas Adams"

3. Diagrama de Sintaxe:

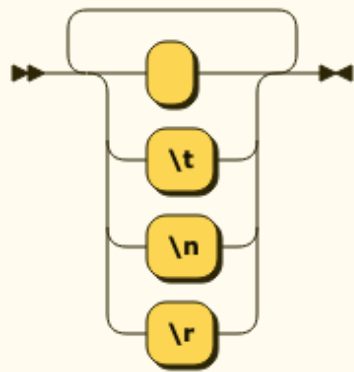
JSON_STRING:



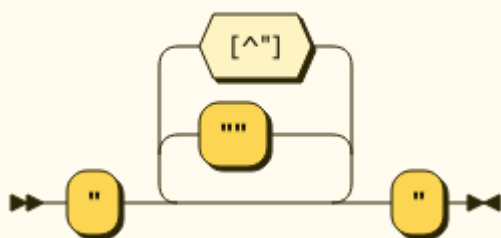
JSON_NUMBER:



JSON_WS:



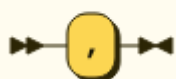
CSV_STRING:



CSV_TEXT:



SEPARATOR:



Conclusão

Conclui-se que a complexidade visual do JSON, especialmente em strings e números, reflete um rigor sintático voltado à integridade e tipagem dos dados já na análise léxica, minimizando ambiguidades. Em contrapartida, o CSV simplifica sua representação através de escapes por duplicação (") e fluxos menos restritivos, o que oferece maior flexibilidade, mas delega a validação para etapas posteriores. Assim, os diagramas revelam que, enquanto o JSON prioriza a confiabilidade e a precisão estrutural ao rejeitar entradas malformadas imediatamente, o CSV foca na simplicidade de representação, aceitando sequências de caracteres de forma mais permissiva.

Referências

ANTLR

LAB.

ANTLR Lab – Parser Interpreter. Disponível em: <http://labantlr.org/>.

BOTTLECAPS.

Railroad Diagram Generator. Disponível em: <https://www.bottlecaps.de/rr/ui>.

TELUS DIGITAL.

An introduction to language lexing and parsing with ANTLR. Disponível em: <https://www.telusdigital.com/insights/digital-experience/article/an-introduction-to-language-lexing-and-parsing-with-antlr>.

SEBESTA, Robert W.

Conceitos de linguagens de programação. 9. ed. São Paulo: Pearson, 2018.

Cap. 4 — análise léxica